

Keywords: diffusion models • reinforcement learning • reward alignment • path-space gradient

Designing Reinforcement Learning for Diffusion Models: A Unified Path-Space View

Importance Sampling Between Sampling SDEs Connects All Major Diffusion-RL Algorithms Under One Principle

By Yixian Xu, Yuanrui Zhang, Shengjie Luo, Liwei Wang, Di He

arXiv:2608.14430 • Peking University; ByteDance Seed, August 2026

Reinforcement learning fine-tuning aligns diffusion models with task-specific rewards, but its algorithmic landscape is fragmented: reverse-trajectory methods, forward-matching methods, and value-gradient approaches each appear to follow different principles. This paper shows they all arise from a single path-space importance-sampling derivation applied to the diffusion SDE. The unification reveals a structured design space whose axes are value-gradient estimation, weight functions, and sampling strategy. From this space the authors derive a multi-sample KDE value-gradient estimator that reduces gradient variance $3.2\times$ and outperforms prior methods by up to 8.3 pp on human preference tasks.

AT A GLANCE

Problem: RL methods for diffusion model alignment are algorithmically fragmented with no shared theoretical basis, preventing principled comparison or design.

Why it matters: A unified framework allows systematic ablation and derivation of new, more efficient algorithms.

Method: Importance sampling between sampling SDEs on trajectory space; derive policy gradient, then its value-gradient form; organize algorithm space by estimation, weight, and sampling choices.

Result: KDE value-gradient estimator cuts gradient variance $3.2\times$ and improves human preference alignment $+8.3$ pp on SD3.5-M.

Evidence: Ablations over all design-space axes; evaluation on image generation (SD3.5-M) and vision-language alignment (Qwen-Image).

Diffusion models define a generative process through a forward SDE that gradually adds noise to data and a reverse SDE that denoises from pure noise to samples. Fine-tuning for reward alignment means biasing the reverse process toward high-reward x_0 without destroying distributional coverage. Reinforcement learning provides the right framework: treat each denoising step as a policy action and optimize expected reward.

But several RL algorithms for diffusion models have been proposed independently, and their relationships are opaque:

- **DDPO/DPO-Diffusion:** Apply PPO-style clipped probability ratios over the full reverse trajectory.
- **AlignProp/ReFL:** Backpropagate reward gradients directly through the reverse SDE.
- **Forward-matching methods:** Train on reward-weighted forward-noised samples.

This paper asks: are these the same algorithm with different approximations, or genuinely different approaches?

Why Existing Approaches Fall Short

Without a unifying theory, practitioners must choose among algorithms without understanding what design choices actually matter. This has several consequences:

Ad hoc hyperparameter selection: Clipping thresholds, sampling schedules, and weight functions are set empirically; small changes produce large performance differences.

High variance: REINFORCE-style estimators applied to the full reverse trajectory have variance that scales with trajectory length. Prior methods apply heuristics to reduce variance without theoretical grounding.

Inaccessible algorithm space: It is unclear whether a new estimator can be derived within this framework or whether improvements require fundamentally new ideas.

Core Mechanism

The paper applies importance sampling between the current reverse-SDE measure $p_\theta(\tau)$ and the reward-weighted measure $p^*(\tau) \propto p_\theta(\tau)R(x_0)$ on the full trajectory space $\tau = (x_T, \dots, x_0)$.

The importance weight is:

$$w(\tau) = \frac{p^*(\tau)}{p_\theta(\tau)} = \frac{R(x_0)}{\mathcal{Z}}$$

where $\mathcal{Z} = \mathbb{E}[R(x_0)]$ normalizes the measure.

The policy gradient $\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{p_\theta}[R(x_0)]$ becomes:

$$\nabla_\theta J = \mathbb{E}_{p_\theta}[R(x_0) \nabla_\theta \log p_\theta(\tau)]$$

The Big Picture

This is a standard policy gradient on trajectory space. Factoring $\log p_\theta(\tau) = \sum_t \log p_\theta(x_{t-1}|x_t)$ shows that each reverse step contributes a per-step gradient weighted by the downstream reward.

Technical Formulation

The **value function** at denoising step t is:

$$V^\pi(x_t) = \mathbb{E}_{p_\theta(\tau_{t:0}|x_t)}[R(x_0)]$$

The policy gradient can be re-expressed in value-gradient form:

$$\nabla_\theta J = \mathbb{E}_{x_T}[\nabla_\theta V^\pi(x_T)]$$

where $x_T \sim \mathcal{N}(0, I)$ is the initial noise.

The paper proposes a **multi-sample KDE value estimator**:

$$\hat{V}(x_t) = \frac{\sum_{i=1}^K R(x_0^{(i)}) K_h(x_0 - x_0^{(i)})}{\sum_{i=1}^K K_h(x_0 - x_0^{(i)})}$$

where K rollouts $\{x_0^{(i)}\}$ are drawn from $p_\theta(\cdot|x_t)$ and a Gaussian kernel K_h smooths the estimate. Variance is:

$$\text{Var}(\hat{V}_{\text{KDE}}) = O(1/K)$$

compared to $O(1)$ for the single-sample REINFORCE baseline.

The three design axes of the unified space are:

1. **Weight function** $f(w(\tau))$: identity (vanilla PG), clip (ϵ -PPO), binary advantage ($\mathbb{I}[R > c]$)
2. **Value-gradient estimation**: direct backprop through SDE, REINFORCE, multi-sample KDE, separate critic
3. **Sampling strategy**: on-policy, off-policy with importance correction, mixed

Existing algorithms correspond to specific combinations; new ones emerge from unexplored combinations.

Learning or Inference Procedure

Training loop:

1. Sample M initial noises $\{x_T^{(m)}\}$
2. For each $x_T^{(m)}$, draw K rollouts of the full reverse SDE to obtain $\{x_0^{(m,k)}\}$
3. Compute reward $R(x_0^{(m,k)})$ for each sample
4. Compute KDE value estimates $\hat{V}(x_t^{(m)})$ at each step
5. Compute the value gradient and apply an optimizer step to θ

Inference: Standard diffusion model sampling; no change to the inference pipeline.

What the Guarantee Says

Under smoothness of R and the reverse SDE transition kernels, the KDE estimator satisfies:

$$\text{Var}(\hat{V}_{\text{KDE}}) \leq \frac{C\|R\|_\infty^2}{Kh^d}$$

where d is the data dimension and h the kernel bandwidth. Bias is $O(h^2)$. The optimal bandwidth balancing bias and variance gives $h = O(K^{-1/(d+4)})$, and the total mean-squared error decays as $O(K^{-4/(d+4)})$.

Experimental Findings

- **SD3.5-M (image generation):** KDE estimator outperforms single-sample REINFORCE by +8.3 pp on human preference alignment; outperforms AlignProp by +4.6 pp.
- **Qwen-Image (VQA alignment):** +4.1 pp absolute over AlignProp on target reward; +2.3 pp over DDPO.
- **Variance reduction:** KDE with $K = 8$ reduces gradient variance $3.2\times$ vs. single-sample; variance continues decreasing to $K = 32$ with diminishing returns.
- **Design space ablations:** Binary advantage weighting degrades performance -3.1 pp vs. identity; off-policy with importance correction recovers +1.8 pp of the gap.

Ablations and Interpretation

Number of rollouts K : The best pass-at-1 reward improves monotonically with K up to $K = 16$; wall-clock time scales linearly. At $K = 8$ the compute-performance trade-off is near-optimal.

Weight function: Identity weighting (vanilla PG) outperforms PPO clipping in the low-data regime; clipping helps stability when off-policy data is reused.

Trajectory length: Methods that apply the gradient estimator only at the final denoising step (equivalent to treating the full reverse trajectory as a black box) underperform full-trajectory methods by -5.2 pp, confirming that per-step value estimates matter.

Softmax saturation (for discrete diffusion variants): The KDE estimator generalizes to discrete diffusion language models with a categorical kernel; variance reduction persists ($2.8\times$).

Reference

Yixian Xu, Yuanrui Zhang, Shengjie Luo, Liwei Wang, Di He. **Designing Reinforcement Learning for Diffusion Models: A Unified Path-Space View.**

arXiv:2608.14430, Peking University; ByteDance Seed,
August 2026. <https://arxiv.org/abs/2608.14430>